

Data Cleaning

1. Load Dataset

In [56]: `import pandas as pd`

```
df = pd.read_csv( r"C:\Users\Vinay Kumar\Downloads\ecommerce_22000_rows.csv")
df.head(20)
```

Out[56]:

	Order_ID	Customer_ID	Product_ID	Category	Original_Price	Discount_%	Quantity
--	----------	-------------	------------	----------	----------------	------------	----------

0	1	1059	239	Clothing	1612.17	20	
1	2	1086	236	Home	702.42	0	
2	3	1050	216	Beauty	1671.69	0	
3	4	1032	243	Home	767.11	5	
4	5	1044	236	Electronics	506.60	10	
5	6	1003	225	Clothing	743.42	5	
6	7	1019	249	Clothing	914.47	10	
7	8	1001	256	Electronics	344.95	20	
8	9	1085	258	Home	1676.03	10	
9	10	1032	243	Electronics	856.51	15	
10	11	1039	222	Electronics	683.46	20	
11	12	1057	209	Clothing	1564.65	15	
12	13	1020	233	Electronics	192.13	15	
13	14	1004	230	Clothing	732.73	10	
14	15	1094	235	Beauty	1570.80	0	
15	16	1098	222	Beauty	376.38	20	
16	17	1023	243	Beauty	1889.44	10	
17	18	1050	203	Home	1236.30	30	
18	19	1012	216	Home	1449.92	0	
19	20	1032	234	Electronics	1729.84	5	



2. Missing Value Treatment

In [57]: `df.isnull().sum()`

```
Out[57]: Order_ID      0
         Customer_ID   0
         Product_ID    0
         Category      0
         Original_Price 0
         Discount_%     0
         Quantity      0
         Order_Date    0
         Final_Price    0
         Revenue       0
         Cost          0
         Profit        0
         Customer_Age   0
         Customer_Gender 0
         Region        0
         Payment_Method 0
         Month          0
         Season         0
         Adjusted_Revenue 0
         dtype: int64
```

3. Column Removal Strategy

```
In [58]: df.columns
```

```
Out[58]: Index(['Order_ID', 'Customer_ID', 'Product_ID', 'Category', 'Original_Price',
               'Discount_%', 'Quantity', 'Order_Date', 'Final_Price', 'Revenue',
               'Cost', 'Profit', 'Customer_Age', 'Customer_Gender', 'Region',
               'Payment_Method', 'Month', 'Season', 'Adjusted_Revenue'],
              dtype='object')
```

```
In [59]: df.drop(columns=['Order_ID', 'Customer_ID'], inplace=True)
```

```
In [60]: df.head(20)
```

Out[60]:

	Product_ID	Category	Original_Price	Discount_%	Quantity	Order_Date	Final_Pric
0	239	Clothing	1612.17	20	3	12-04-2024	1289.7
1	236	Home	702.42	0	2	11-03-2025	702.4
2	216	Beauty	1671.69	0	2	10-05-2026	1671.6
3	243	Home	767.11	5	4	27-09-2024	728.7
4	236	Electronics	506.60	10	4	16-04-2024	455.9
5	225	Clothing	743.42	5	3	12-03-2024	706.2
6	249	Clothing	914.47	10	1	01-12-2025	823.0
7	256	Electronics	344.95	20	1	21-01-2024	275.9
8	258	Home	1676.03	10	3	06-09-2025	1508.4
9	243	Electronics	856.51	15	1	01-05-2024	728.0
10	222	Electronics	683.46	20	4	11-04-2025	546.7
11	209	Clothing	1564.65	15	3	02-08-2024	1329.9
12	233	Electronics	192.13	15	4	26-11-2024	163.3
13	230	Clothing	732.73	10	1	03-04-2025	659.4
14	235	Beauty	1570.80	0	2	28-03-2024	1570.8
15	222	Beauty	376.38	20	4	07-01-2025	301.1
16	243	Beauty	1889.44	10	1	09-04-2024	1700.5
17	203	Home	1236.30	30	2	21-05-2026	865.4
18	216	Home	1449.92	0	3	25-10-2025	1449.9
19	234	Electronics	1729.84	5	4	10-05-2024	1643.3

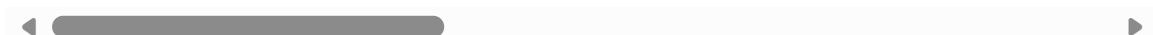


In [85]: `df = pd.read_csv( r"C:\Users\Vinay Kumar\Downloads\ecommerce_22000_rows.csv")`

In [86]: `df.head()`

Out[86]:

	Order_ID	Customer_ID	Product_ID	Category	Original_Price	Discount_%	Quantity
0	1	1059	239	Clothing	1612.17	20	3
1	2	1086	236	Home	702.42	0	2
2	3	1050	216	Beauty	1671.69	0	2
3	4	1032	243	Home	767.11	5	4
4	5	1044	236	Electronics	506.60	10	4



4. Data Type Conversion

```
In [87]: df.dtypes
```

```
Out[87]: Order_ID          int64
Customer_ID        int64
Product_ID         int64
Category           object
Original_Price     float64
Discount_%         int64
Quantity           int64
Order_Date         object
Final_Price        float64
Revenue            float64
Cost               float64
Profit             float64
Customer_Age       int64
Customer_Gender    object
Region            object
Payment_Method     object
Month              int64
Season             object
Adjusted_Revenue   float64
dtype: object
```

```
In [88]: df['Category'] = df['Category'].astype('category')
```

```
In [89]: df['Discount_%'] = df['Discount_%'].astype(float)
```

```
In [90]: df['Order_Date'] = pd.to_datetime(
    df['Order_Date'],
    format='%d-%m-%Y'
)
```

5. Text Standardization

```
In [91]: for col in ['Order_ID', 'Customer_ID', 'Product_ID', 'Category', 'Original_Price',
    'Discount_%', 'Quantity', 'Order_Date', 'Final_Price', 'Revenue',
    'Cost', 'Profit', 'Customer_Age', 'Customer_Gender', 'Region',
    'Payment_Method', 'Month', 'Season', 'Adjusted_Revenue']:
    print(f"\n{col} unique values:")
    print(df[col].unique())
```

Order_ID unique values:

```
[ 1 2 3 ... 21998 21999 22000]
```

Customer_ID unique values:

```
[1059 1086 1050 1032 1044 1003 1019 1001 1085 1039 1057 1020 1004 1094
 1098 1023 1012 1006 1071 1072 1029 1000 1096 1091 1051 1067 1088 1063
 1095 1084 1073 1018 1046 1017 1024 1078 1043 1037 1016 1052 1025 1042
 1081 1066 1061 1002 1074 1045 1076 1075 1077 1013 1089 1092 1011 1014
 1083 1026 1009 1007 1015 1031 1079 1040 1021 1068 1038 1070 1053 1082
 1047 1087 1008 1069 1041 1035 1097 1056 1060 1090 1005 1054 1028 1022
 1055 1099 1093 1064 1036 1062 1033 1065 1049 1027 1080 1034 1048 1030
 1058 1010]
```

Product_ID unique values:

```
[239 236 216 243 225 249 256 258 222 209 233 230 235 203 234 214 205 229
 211 244 238 254 208 232 207 241 217 237 231 201 210 242 212 257 253 251
 223 252 259 248 202 226 246 213 206 200 255 245 250 228 224 240 227 221
 219 215 247 220 218 204]
```

Category unique values:

```
['Clothing', 'Home', 'Beauty', 'Electronics']
```

```
Categories (4, object): ['Beauty', 'Clothing', 'Electronics', 'Home']
```

Original_Price unique values:

```
[1612.17 702.42 1671.69 ... 1056.08 1941.31 1161.69]
```

Discount_% unique values:

```
[20. 0. 5. 10. 15. 30.]
```

Quantity unique values:

```
[3 2 4 1]
```

Order_Date unique values:

```
<DatetimeArray>
```

```
['2024-04-12 00:00:00', '2025-03-11 00:00:00', '2026-05-10 00:00:00',
 '2024-09-27 00:00:00', '2024-04-16 00:00:00', '2024-03-12 00:00:00',
 '2025-12-01 00:00:00', '2024-01-21 00:00:00', '2025-09-06 00:00:00',
 '2024-05-01 00:00:00',
 ...
 '2025-08-22 00:00:00', '2025-03-08 00:00:00', '2024-07-15 00:00:00',
 '2025-11-24 00:00:00', '2024-10-25 00:00:00', '2024-02-21 00:00:00',
 '2024-03-11 00:00:00', '2026-02-03 00:00:00', '2025-11-01 00:00:00',
 '2026-03-18 00:00:00']
```

```
Length: 1000, dtype: datetime64[ns]
```

Final_Price unique values:

```
[1289.74 702.42 1671.69 ... 813.18 1003.47 1619.98]
```

Revenue unique values:

```
[3869.22 1404.85 3343.39 ... 3010.41 470.23 1619.98]
```

Cost unique values:

```
[ 914.04 482.77 1250.97 563.3 408.72 467.54 646.12 173.58 1239.19
 650.69 544.74 846.55 116.14 571.58 1205.6 243.65 1105.93 956.89
 881.9 1235.38 561.16 1185.94 1006.14 1000.56 706.82 188.2 217.86
 618.86 636.4 238. 279.28 515.21 410.53 616.48 745.36 136.58
 596.56 140.35 483.24 1077.75 690.42 809.8 729.76 523.78 553.27
 632.65 548.39 192.71 179.38 1100.74 318.49 182.57 379.82 324.75
 311.79 871.4 1115.33 729.34 943.45 843.83 1026.69 1194.03 1351.85
 61.66 533.81 975.89 97.6 200.22 798.62 639.97 364.19 870.16]
```

1305.47	896.56	1232.58	1126.91	158.76	1203.22	534.61	531.84	881.62
648.69	565.45	539.19	926.97	1040.09	725.88	694.64	1265.17	918.89
1423.86	1056.22	144.2	170.79	414.89	687.47	949.15	134.27	322.73
932.87	406.84	330.07	168.57	996.05	774.2	753.92	120.19	1366.09
71.88	730.2	348.21	228.26	1181.82	735.41	309.8	896.45	1052.51
214.09	450.83	331.85	151.31	103.15	172.48	540.19	266.88	84.36
1107.12	596.04	1158.56	362.84	1448.21	1393.14	1210.39	1028.79	81.73
1026.54	861.05	318.86	868.66	629.84	518.81	527.46	81.77	667.52
221.55	1093.37	911.57	799.25	326.35	421.2	1194.62	499.15	137.01
918.56	798.99	357.54	145.51	794.14	1096.25	868.89	364.25	1352.05
904.45	839.2	650.27	712.75	1050.99	1063.66	707.44	716.49	634.29
235.3	963.77	66.82	935.35	109.86	1086.28	143.41	1276.91	161.57
213.58	295.21	881.22	1478.22	970.96	1059.68	912.78	798.56	211.86
948.33	84.22	181.71	313.68	243.06	441.1	92.28	179.62	1310.95
281.11	1048.97	744.29	847.17	1244.76	1149.65	1381.94	216.76	96.48
926.05	1174.18	945.23	757.52	1321.95	82.97	240.77	1019.52	642.98
681.51	732.06	69.79	454.75	710.12	584.34	343.66	72.23	716.13
230.45	566.63	267.46	606.1	641.4	416.92	133.	438.88	220.94
1222.8	910.87	510.04	417.26	1055.76	369.39	824.5	594.07	986.79
232.71	549.63	472.92	576.01	944.08	1003.83	910.09	1111.53	1303.98
902.08	611.93	683.81	1098.76	298.42	1183.05	913.84	618.24	662.12
1028.28	698.72	702.26	119.3	795.07	1022.14	848.57	598.65	1245.38
223.22	420.37	824.41	1056.4	959.29	1336.57	404.03	797.24	579.7
194.64	1035.33	838.88	826.02	886.93	263.1	207.77	719.67	246.61
614.98	1294.71	369.2	753.94	1030.99	115.17	127.14	334.05	85.31
1008.31	773.36	89.87	186.29	708.17	1124.35	911.04	966.38	452.4
540.99	1017.02	95.43	199.43	1262.93	1258.22	1348.63	258.98	869.95
574.08	146.63	868.01	1381.47	1173.87	202.5	276.96	805.09	68.06
518.47	494.43	883.71	612.28	775.42	885.03	538.57	538.5	391.65
300.22	1326.14	264.02	667.76	683.97	363.77	1291.24	594.39	1057.45
68.79	896.53	817.72	1116.11	513.02	499.35	603.74	306.86	1303.05
567.86	427.13	225.37	829.22	515.04	666.06	256.11	544.49	769.69
1445.1	1161.25	891.71	1203.01	896.04	1205.18	393.75	370.54	805.13
841.56	1196.37	1387.49	1025.41	1391.27	121.98	383.39	551.08	901.9
967.76	497.47	740.	789.53	611.76	667.88	686.57	1164.88	673.39
330.24	1156.41	1117.89	696.51	840.21	1056.43	1426.95	435.21	1100.73
314.35	587.37	431.42	818.78	575.59	395.89	1344.8	1364.67	754.12
815.94	335.63	553.2	701.23	995.19	746.12	1198.51	866.55	1338.77
549.98	288.04	1080.51	325.74	1116.1	987.61	797.96	937.48	1085.09
937.31	827.19	282.01	1440.69	962.87	493.7	278.02	1455.12	543.63
1044.7	505.5	198.86	1014.78	247.73	981.98	561.71	145.36	66.88
1571.67	99.06	1286.91	1326.56	1055.78	360.4	1061.09	172.79	1048.5
867.02	150.97	514.1	233.72	1428.48	114.01	655.02	1297.37	972.19
1361.38	994.33	801.95	207.74	644.95	362.55	113.5	274.09	554.9
108.08	1250.08	303.78	571.94	1022.42	803.14	868.39	825.71	539.32
630.7	866.4	458.22	1227.34	767.47	604.25	368.32	711.51	324.26
758.98	669.35	461.52	1266.26	1511.72	879.04	1168.38	1381.51	316.9
232.02	106.21	417.32	98.2	1133.72	139.16	405.21	285.53	243.82
151.72	598.88	770.75	832.83	865.92	99.45	429.58	669.95	162.03
739.49	997.75	137.16	549.47	668.43	835.84	454.43	702.21	196.89
321.15	415.97	205.4	1364.33	847.81	591.33	1384.13	1229.59	470.86
120.23	302.9	1119.83	1014.35	624.4	590.85	1329.97	873.65	701.41
1128.08	464.02	302.42	997.17	1044.26	184.54	894.12	870.58	377.74
598.8	271.5	416.02	1064.95	1081.7	541.42	360.71	717.78	340.12
983.83	139.06	119.14	312.98	896.06	511.8	136.47	1380.94	346.82
154.95	429.31	129.68	1000.03	1270.7	235.24	1102.27	629.08	777.91
1134.05	1219.24	544.01	559.72	183.08	173.84	1016.29	191.89	1048.41
903.09	885.11	888.22	1015.48	1002.54	865.99	1173.88	948.6	954.19
1048.15	234.24	1389.19	336.58	1123.05	645.66	634.15	156.47	1207.75
227.92	365.81	782.36	894.65	331.72	495.28	896.48	918.91	273.67

162.3	756.22	1290.12	925.94	52.54	1537.29	521.66	504.93	673.02
985.49	425.73	1271.82	1305.73	231.57	1292.55	404.38	377.28	811.04
1015.26	350.36	724.57	168.47	288.09	117.37	945.38	843.44	324.21
1159.	311.03	284.05	168.4	352.58	369.69	1233.9	711.97	965.69
1067.7	1251.73	1174.7	141.32	900.6	310.45	331.53	273.11	475.03
151.91	544.33	1258.02	585.67	184.33	656.6	845.15	868.83	246.33
325.91	1025.77	701.45	703.03	604.83	1071.17	998.54	535.26	810.95
521.64	630.74	949.98	210.36	1278.7	86.49	627.94	1286.55	902.48
280.35	638.32	1002.73	462.82	1149.05	94.2	737.66	822.28	685.05
1061.37	982.95	1013.37	455.37	257.68	97.37	1189.61	1359.99	562.55
466.94	1052.41	693.48	451.27	217.46	459.05	412.92	423.73	930.54
634.57	756.44	1154.45	178.21	1144.61	662.64	173.06	795.4	569.92
1135.55	1228.74	982.76	505.66	959.4	909.06	823.94	1115.24	112.13
243.84	348.95	1057.74	450.17	170.66	282.05	852.66	734.43	793.98
121.47	330.06	682.19	684.81	1267.56	1102.19	849.08	1122.18	621.16
808.85	907.41	392.6	730.37	637.64	1536.94	1166.55	420.2	278.9
854.74	490.15	209.07	92.41	391.47	732.69	325.07	484.09	159.05
646.26	1139.92	972.53	953.11	364.54	1167.48	1099.92	1236.71	782.17
559.46	216.94	215.42	1357.04	729.48	1280.06	1302.25	433.46	795.5
208.77	684.71	239.63	1244.89	867.25	731.55	259.3	285.52	703.36
208.24	586.09	104.78	1275.79	883.51	450.13	996.63	1066.84	526.23
829.8	405.08	84.21	1070.04	467.55	740.31	1190.08	435.02	315.
92.29	1316.79	526.37	713.29	336.76	84.42	557.7	1217.16	175.34
682.97	707.28	678.28	600.05	1063.71	772.43	321.52	77.29	942.32
814.32	87.91	1136.16	278.99	857.63	921.96	471.89	940.09	990.39
847.53	1005.47	292.04	910.77	109.14	212.17	774.38	498.26	617.83
233.12	1121.24	676.52	145.12	890.07	475.08	220.17	750.47	1024.04
358.44	1117.33	688.91	1051.72	1180.51	410.26	1072.96	254.73	853.77
144.28	220.37	909.96	508.35	730.8	841.72	226.74	244.78	94.04
206.55	297.45	764.86	462.41	944.77	505.97	645.55	1161.12	995.41
927.29	500.92	271.33	1480.59	480.17	742.15	898.81	591.28	160.24
1127.08	730.32	336.75	240.35	725.33	378.91	1476.95	102.	909.18
62.73	1050.5	504.19	579.5	363.98	589.48	982.1	355.04	399.94
188.42	1160.22	1095.75	669.38	948.31	202.78	563.49	447.64	1280.82
1092.85	348.07	211.25	672.47	546.34	946.17	553.28	630.36	475.39
319.52	209.56	576.6	397.88	769.32	105.27	1307.77	483.3	85.81
1277.23	735.19	385.62	787.46	935.62	337.41	270.84	941.77	245.44
867.57	229.7	433.18	604.03	1230.12	566.77	1211.37	294.34	614.44
67.81	1258.32	421.75	1432.12	752.59	885.39	1364.58	935.05	77.5
1243.23	633.25	1363.99	934.13	1102.85	392.75	1110.23	542.15	916.75
636.12	1001.91	373.51	617.16	72.99	1055.2	353.35	124.24	735.48
1376.32	805.19	1053.28	106.78	936.05	410.09	879.06	582.58	563.23
1042.95	823.28	451.1	213.11	783.82]				

Profit unique values:

[1218.48 429.01 789.12 ... 334.82 561.72 720.59]

Customer_Age unique values:

[26 43 29 23 47 45 33 22 61 62 58 59 49 54 19 32 27 37 21 56 57 31 52 28
20 36 34 42 53 60 41 30 63 24 38 64 44 35 46 40 55 48 51 50 18 39 25]

Customer_Gender unique values:

['Female' 'Male']

Region unique values:

['South' 'West' 'North' 'East']

Payment_Method unique values:

['Debit Card' 'Cash' 'UPI' 'Credit Card']

Month unique values:

```
[ 4  3  5  9 12  1  8 11 10  2  7  6]
```

Season unique values:

```
['Regular' 'Festive']
```

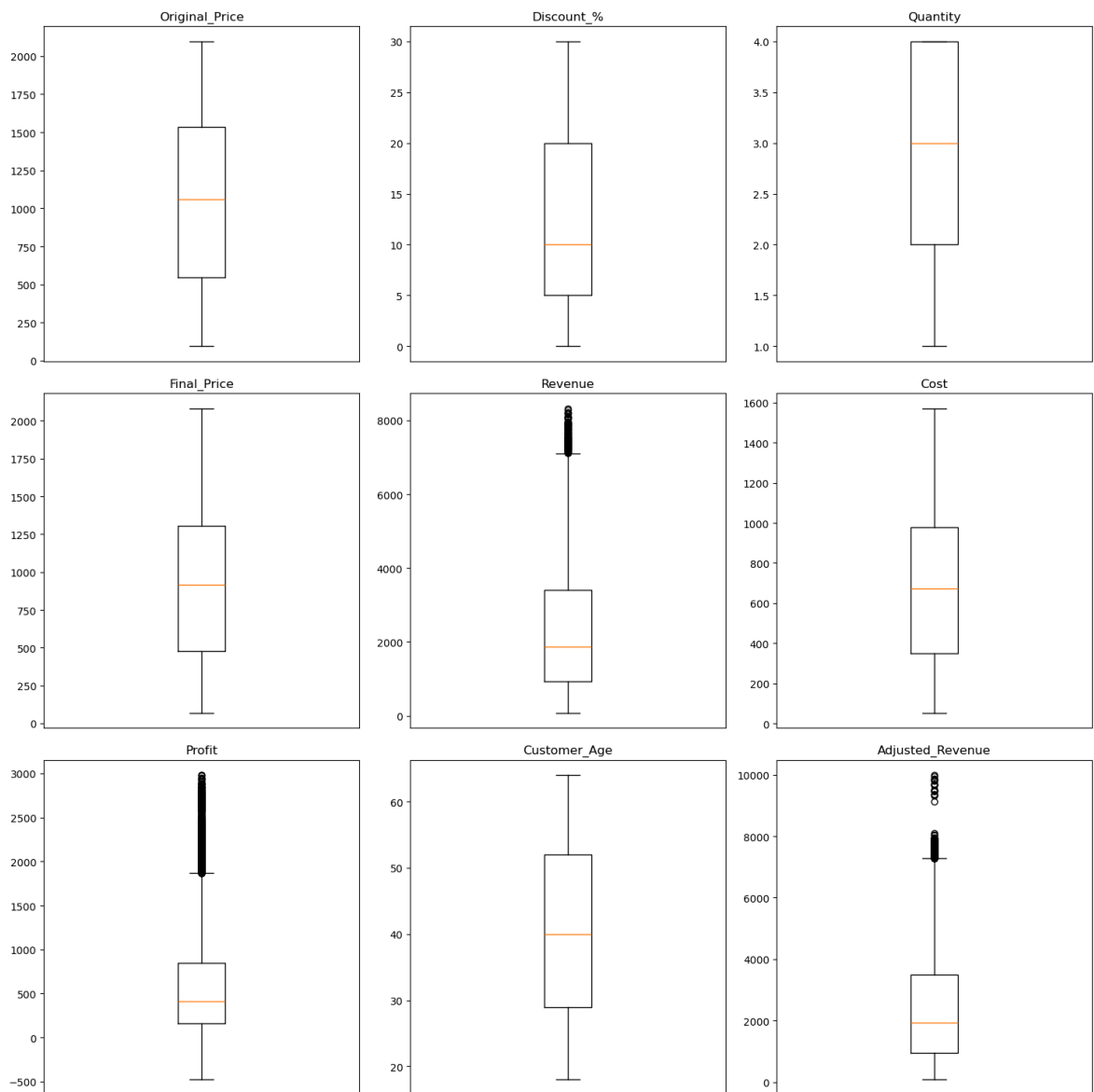
Adjusted_Revenue unique values:

```
[3869.22 1404.85 3343.39 ... 3010.41  470.23 1943.98]
```

6. Outlier Assessment

```
In [92]: num_cols = [  
    'Original_Price', 'Discount_%', 'Quantity',  
    'Final_Price', 'Revenue', 'Cost', 'Profit',  
    'Customer_Age', 'Adjusted_Revenue'  
]
```

```
In [93]: import matplotlib.pyplot as plt  
import math  
  
n = len(num_cols)  
  
cols = 3  
rows = math.ceil(n / cols)  
  
plt.figure(figsize=(15, 5 * rows))  
  
for i, col in enumerate(num_cols, 1):  
    plt.subplot(rows, cols, i)  
    plt.boxplot(df[col])  
    plt.title(f'{col}')  
    plt.xticks([])  
  
plt.tight_layout()  
plt.show()
```

```
In [94]: for col in num_cols:
          Q1 = df[col].quantile(0.25)
          Q3 = df[col].quantile(0.75)
          IQR = Q3 - Q1

          lower_bound = Q1 - 1.5 * IQR
          upper_bound = Q3 + 1.5 * IQR

          outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

          print(f"{col}: {len(outliers)} outliers")
```

```
Original_Price: 0 outliers
Discount_%. 0 outliers
Quantity: 0 outliers
Final_Price: 0 outliers
Revenue: 136 outliers
Cost: 0 outliers
Profit: 1148 outliers
Customer_Age: 0 outliers
Adjusted_Revenue: 220 outliers
```

```
In [95]: #Apply IQR Method + Capping
```

```
In [96]: for col in num_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df[col] = df[col].clip(lower_bound, upper_bound)
```

```
In [97]: #Verify Outliers Are Removed
```

```
In [98]: for col in num_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

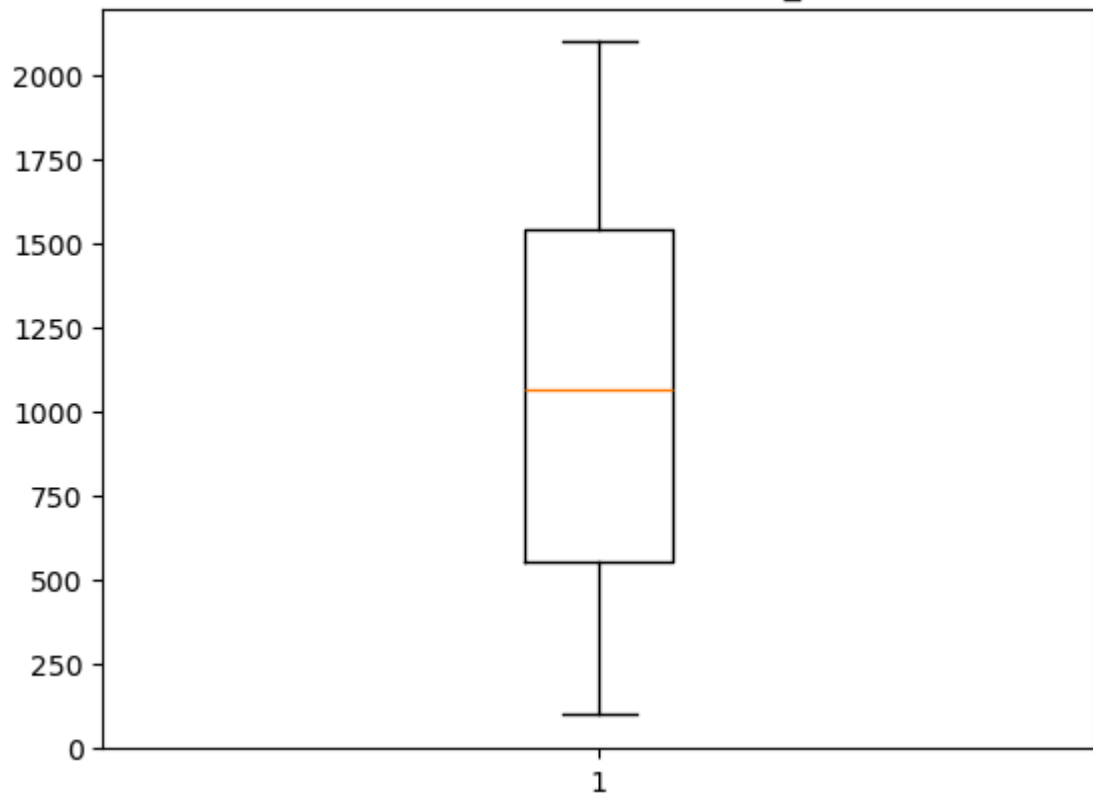
        print(f"{col}: {len(outliers)} outliers remaining")
```

Original_Price: 0 outliers remaining
Discount_#: 0 outliers remaining
Quantity: 0 outliers remaining
Final_Price: 0 outliers remaining
Revenue: 0 outliers remaining
Cost: 0 outliers remaining
Profit: 0 outliers remaining
Customer_Age: 0 outliers remaining
Adjusted_Revenue: 0 outliers remaining

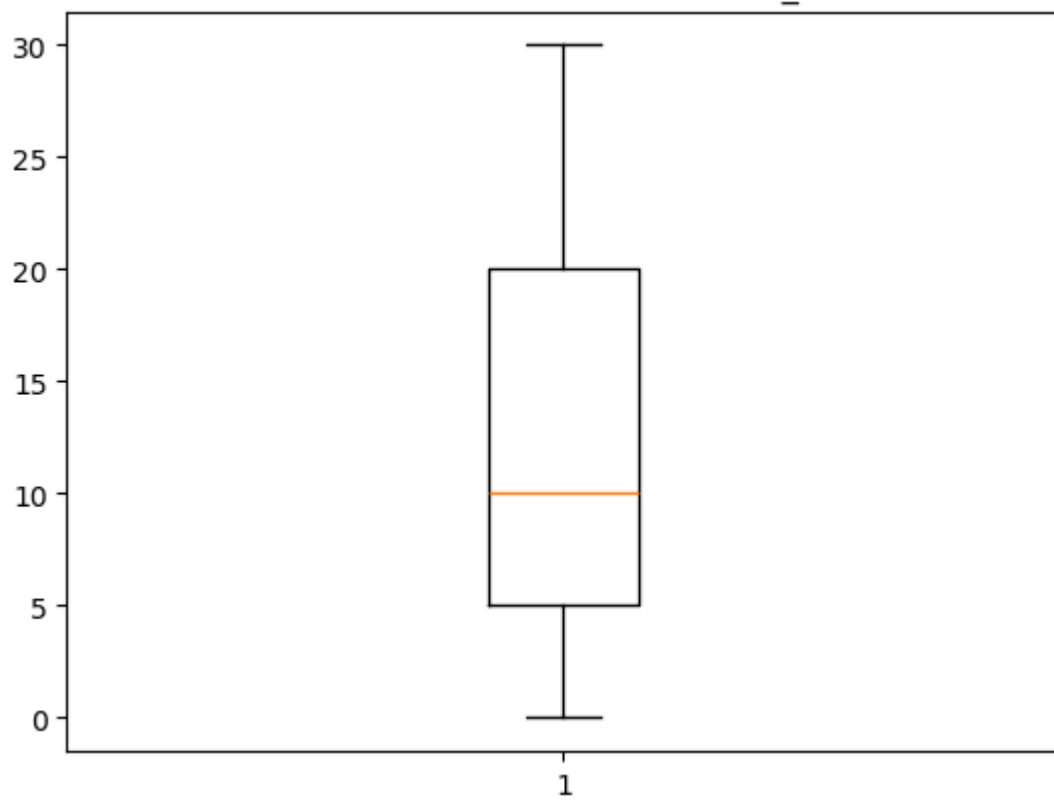
```
In [99]: import matplotlib.pyplot as plt

        for col in num_cols:
            plt.figure()
            plt.boxplot(df[col])
            plt.title(f'After Winsorization: {col}')
            plt.show()
```

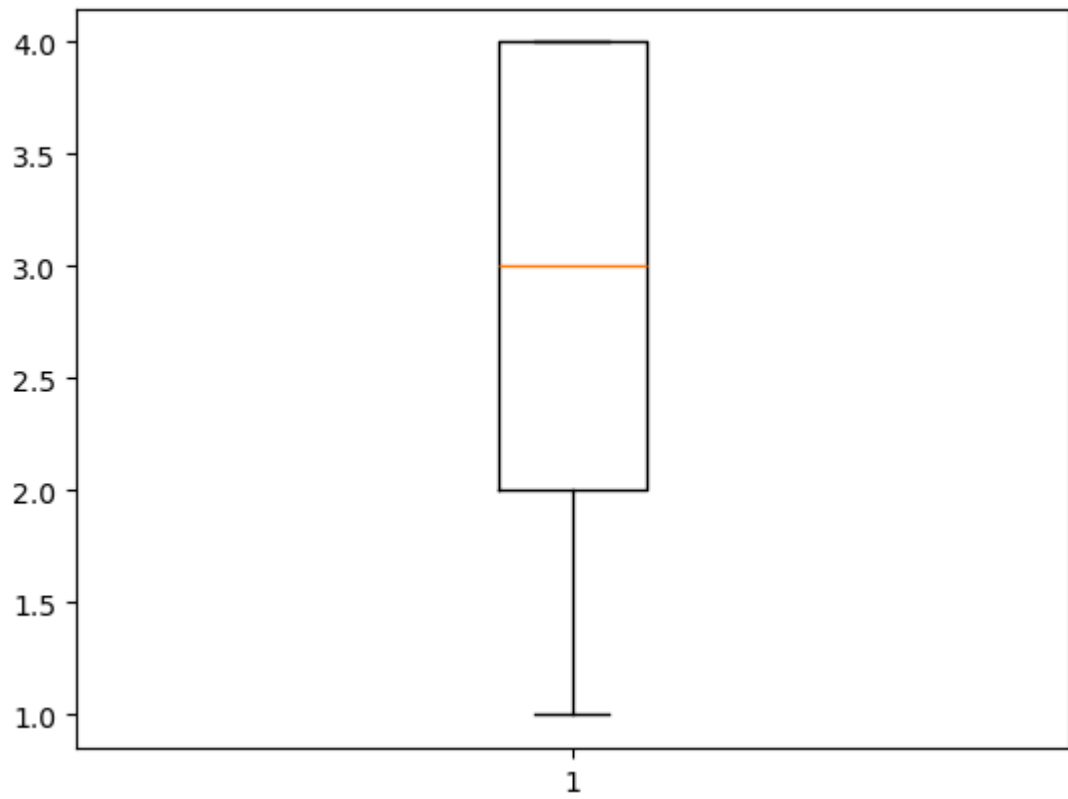
After Winsorization: Original_Price



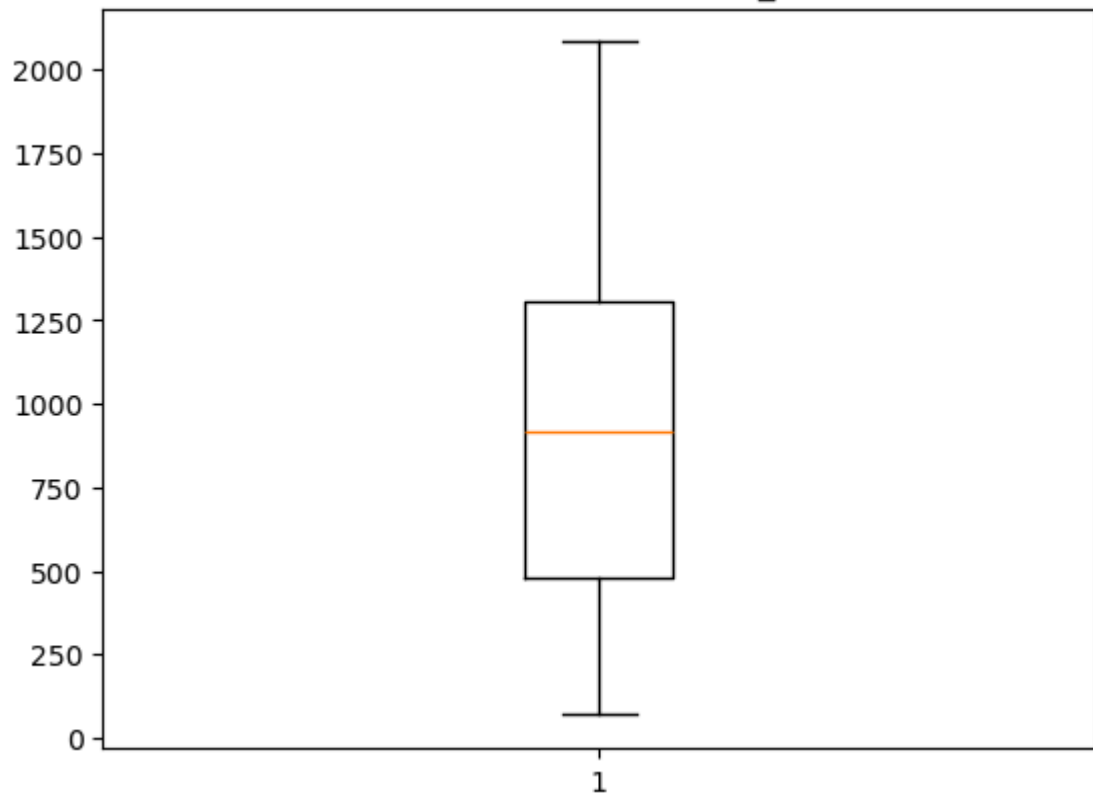
After Winsorization: Discount_%



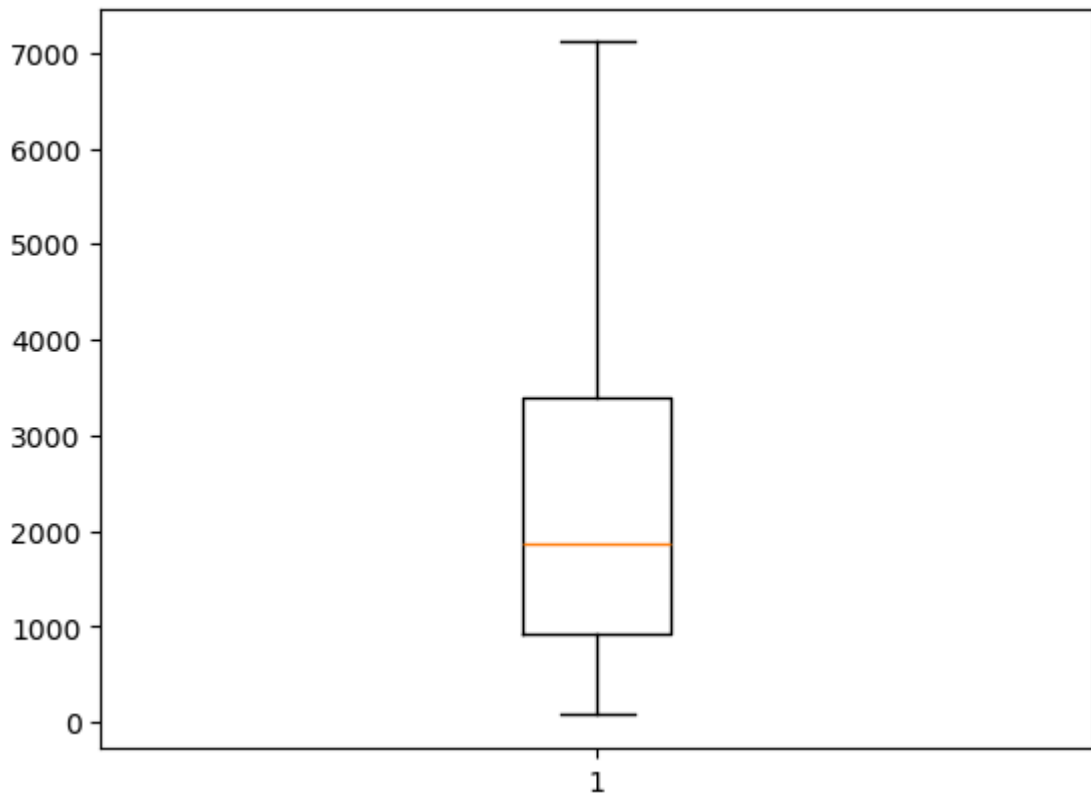
After Winsorization: Quantity



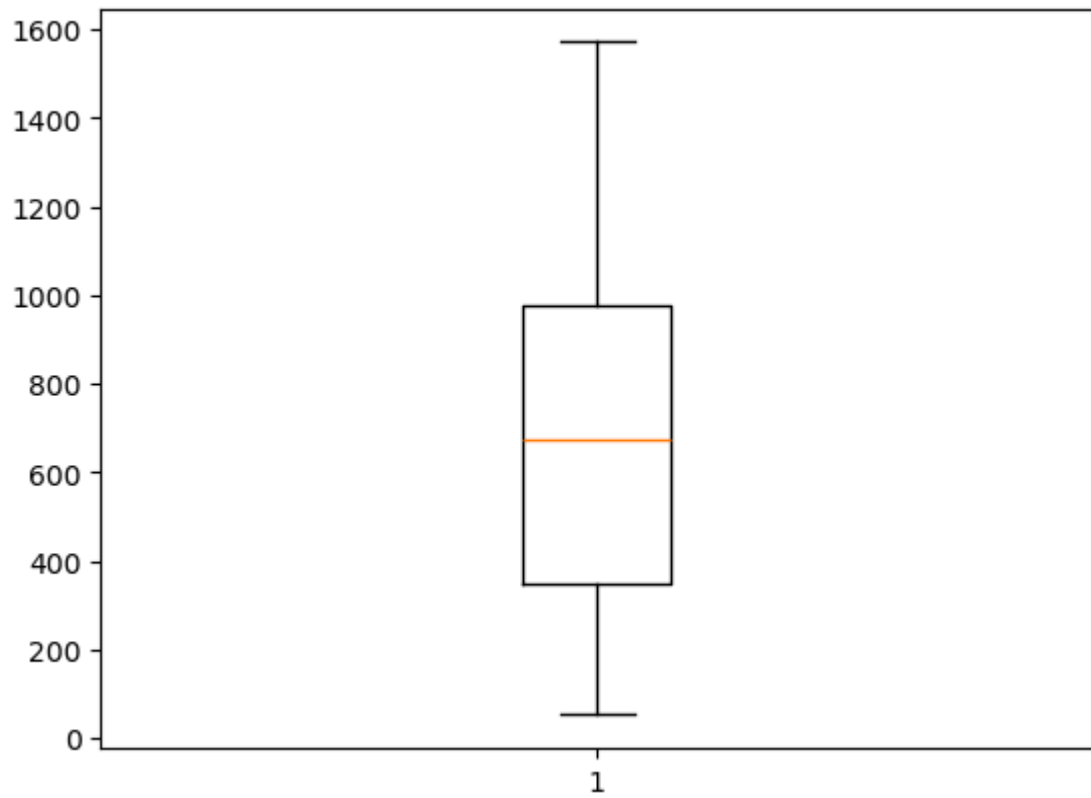
After Winsorization: Final_Price



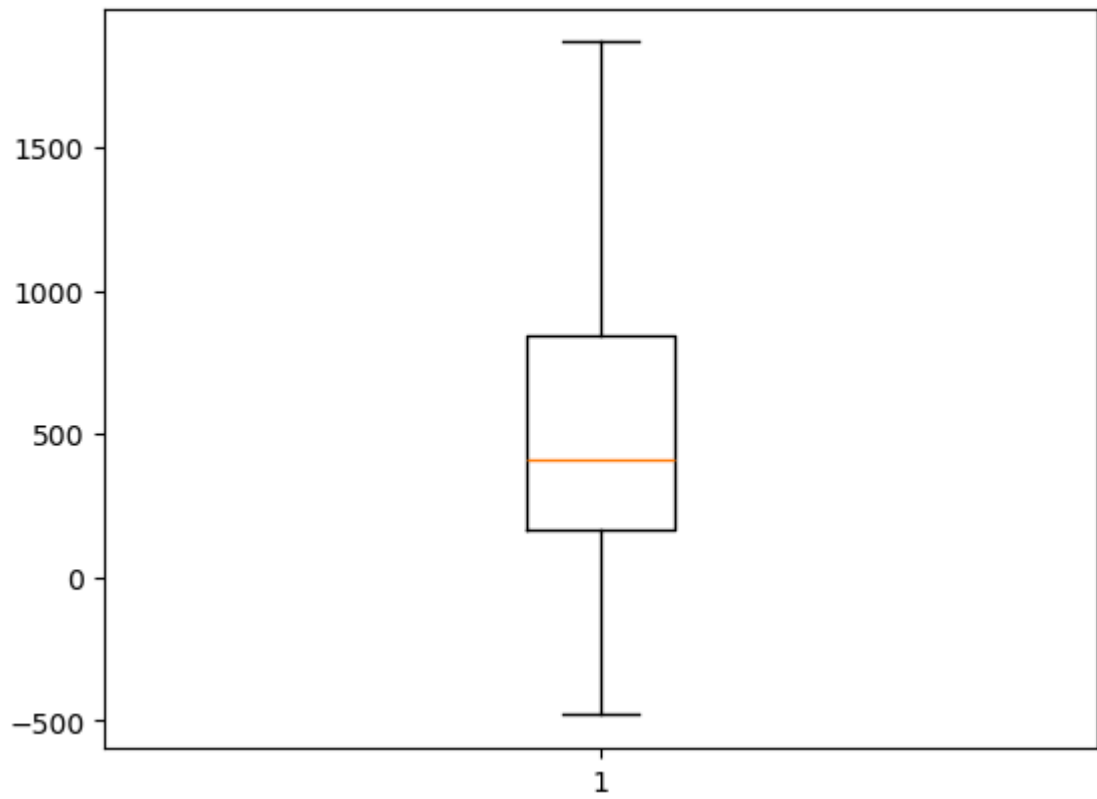
After Winsorization: Revenue



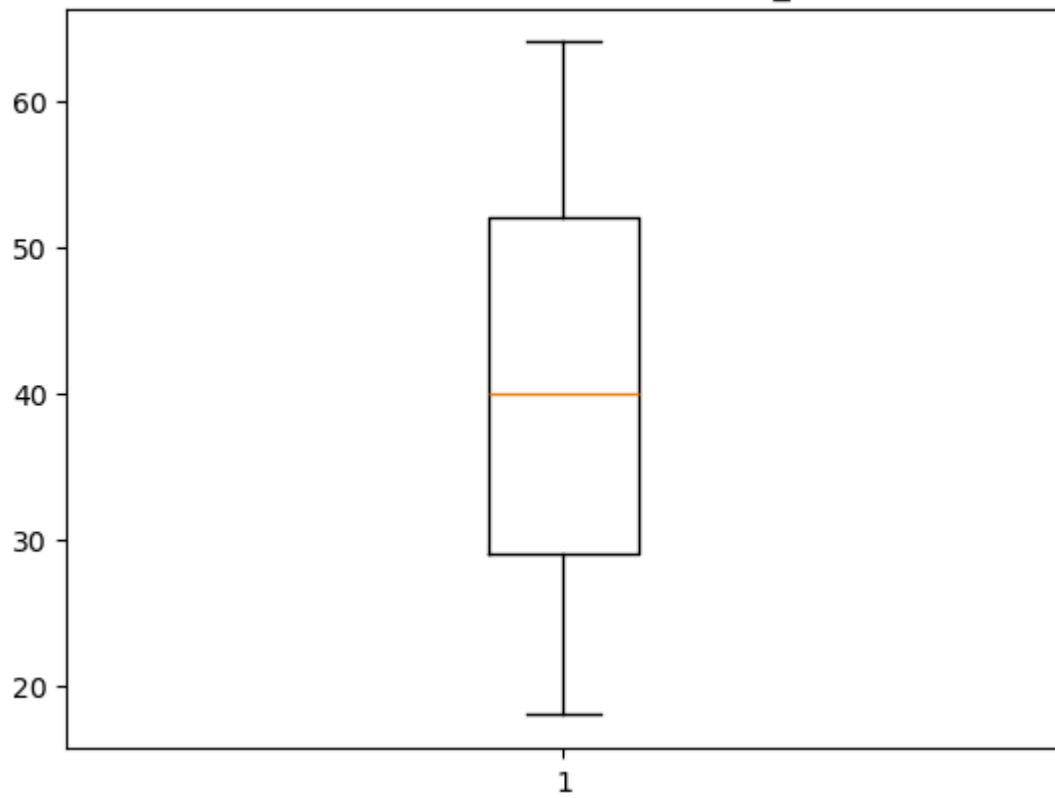
After Winsorization: Cost

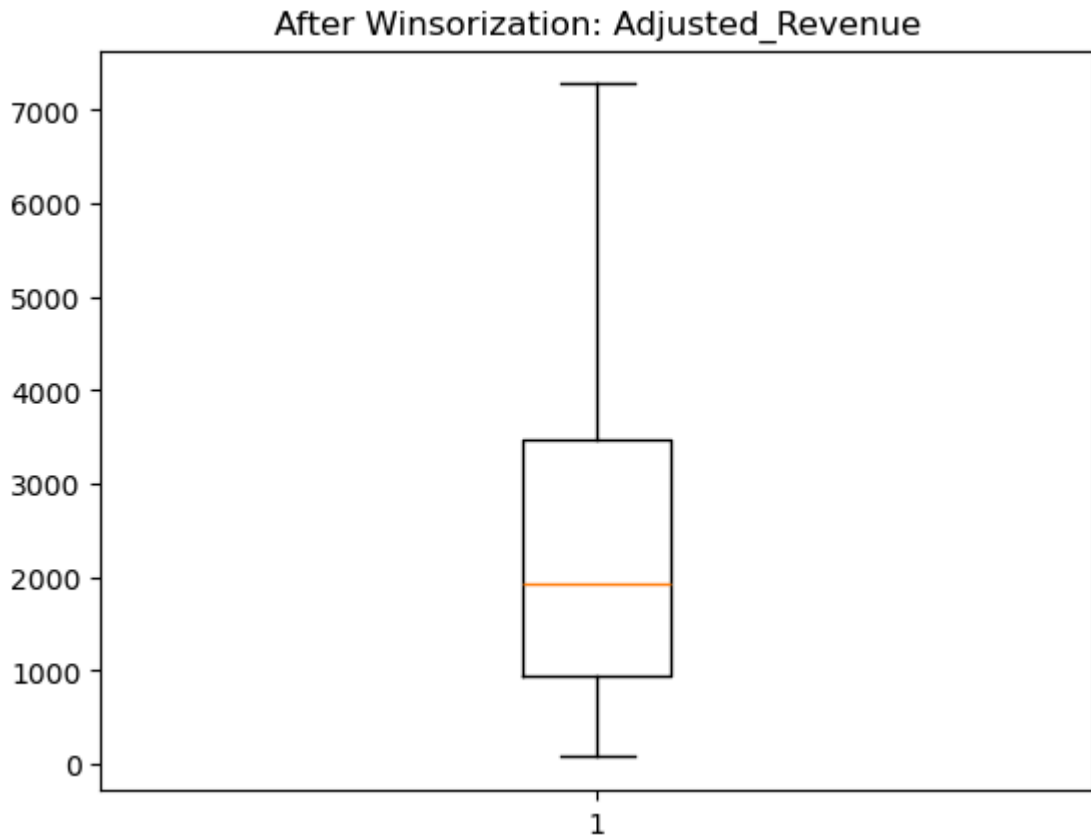


After Winsorization: Profit



After Winsorization: Customer_Age





7. Data Consistency Checks

checking whether calculated fields such as Revenue and Profit match their expected formulas.

1. Revenue Validation

Formula:

Revenue = Final_Price × Quantity

```
In [100... df['Revenue_Check'] = df['Final_Price'] * df['Quantity']

# Compare actual vs calculated
revenue_mismatch = df[df['Revenue'] != df['Revenue_Check']]

print("Revenue mismatches:", len(revenue_mismatch))
```

Revenue mismatches: 11355

2. Profit Validation

Formula:

Profit = Revenue – Cost

```
In [101... df['Profit_Check'] = df['Revenue'] - df['Cost']

# Compare actual vs calculated
profit_mismatch = df[df['Profit'] != df['Profit_Check']]

print("Profit mismatches:", len(profit_mismatch))
```

Profit mismatches: 22000

```
In [102... df['Revenue'] = df['Final_Price'] * df['Quantity']
df['Profit'] = df['Revenue'] - df['Cost']
```

```
In [103... print("Final Revenue mismatches:", len(df[df['Revenue'] != df['Final_Price'] * d
print("Final Profit mismatches:", len(df[df['Profit'] != df['Revenue'] - df['Cos
```

Final Revenue mismatches: 0

Final Profit mismatches: 0

8. Final Clean Dataset

```
In [104... # Check missing values
print(df.isnull().sum())

# Check duplicates
print("Duplicates:", df.duplicated().sum())

# Final data types
print(df.dtypes)

# Preview clean dataset
df.head()
```



```

Order_ID      0
Customer_ID   0
Product_ID    0
Category      0
Original_Price 0
Discount_%    0
Quantity      0
Order_Date    0
Final_Price   0
Revenue       0
Cost          0
Profit        0
Customer_Age  0
Customer_Gender 0
Region        0
Payment_Method 0
Month         0
Season        0
Adjusted_Revenue 0
Revenue_Check 0
Profit_Check  0
dtype: int64
Duplicates: 0
Order_ID      int64
Customer_ID    int64
Product_ID     int64
Category       category
Original_Price float64
Discount_%     float64
Quantity       int64
Order_Date     datetime64[ns]
Final_Price    float64
Revenue        float64
Cost           float64
Profit         float64
Customer_Age   int64
Customer_Gender object
Region         object
Payment_Method object
Month          int64
Season         object
Adjusted_Revenue float64
Revenue_Check  float64
Profit_Check   float64
dtype: object

```

Out[104...

	Order_ID	Customer_ID	Product_ID	Category	Original_Price	Discount_%	Quantity
0	1	1059	239	Clothing	1612.17	20.0	3
1	2	1086	236	Home	702.42	0.0	2
2	3	1050	216	Beauty	1671.69	0.0	2
3	4	1032	243	Home	767.11	5.0	4
4	5	1044	236	Electronics	506.60	10.0	4

5 rows × 21 columns



```
In [105... df.to_csv("cleaned_data.csv", index=False)
```

```
In [106... import os
file_path = os.path.abspath("cleaned_data.csv")
print(file_path)
```

C:\Users\Vinay Kumar\cleaned_data.csv

```
In [107... print(df.shape)      # rows, columns
print(df.head())      # preview
print(df.columns)      # check new columns added
```

(22000, 21)

	Order_ID	Customer_ID	Product_ID	Category	Original_Price	Discount_%	\
0	1	1059	239	Clothing	1612.17	20.0	
1	2	1086	236	Home	702.42	0.0	
2	3	1050	216	Beauty	1671.69	0.0	
3	4	1032	243	Home	767.11	5.0	
4	5	1044	236	Electronics	506.60	10.0	

	Quantity	Order_Date	Final_Price	Revenue	...	Profit	Customer_Age	\
0	3	2024-04-12	1289.74	3869.22	...	2955.18	26	
1	2	2025-03-11	702.42	1404.84	...	922.07	43	
2	2	2026-05-10	1671.69	3343.38	...	2092.41	29	
3	4	2024-09-27	728.76	2915.04	...	2351.74	23	
4	4	2024-04-16	455.95	1823.80	...	1415.08	47	

	Customer_Gender	Region	Payment_Method	Month	Season	Adjusted_Revenue	\
0	Female	South	Debit Card	4	Regular	3869.22	
1	Female	West	Debit Card	3	Regular	1404.85	
2	Male	West	Debit Card	5	Regular	3343.39	
3	Male	South	Cash	9	Regular	2915.04	
4	Male	South	Cash	4	Regular	1823.78	

	Revenue_Check	Profit_Check
0	3869.22	2955.18
1	1404.84	922.08
2	3343.38	2092.42
3	2915.04	2351.74
4	1823.80	1415.06

[5 rows x 21 columns]

```
Index(['Order_ID', 'Customer_ID', 'Product_ID', 'Category', 'Original_Price',
      'Discount_%', 'Quantity', 'Order_Date', 'Final_Price', 'Revenue',
      'Cost', 'Profit', 'Customer_Age', 'Customer_Gender', 'Region',
      'Payment_Method', 'Month', 'Season', 'Adjusted_Revenue',
      'Revenue_Check', 'Profit_Check'],
      dtype='object')
```

```
In [1]: import os
file_path = os.path.abspath("cleaned_data.csv")
print(file_path)
```

C:\Users\Vinay Kumar\cleaned_data.csv

```
In [ ]:
```